

Problema 1 – Aprobarea unui credit bancar (decizie boolean)

Context:

O bancă analizează cererile de credit folosind mai mulți verificatori (responsabili), fiecare ocupându-se de un criteriu specific.

Cerință:

Să se implementeze un sistem bazat pe **Chain of Responsibility** care decide dacă o persoană este eligibilă pentru un credit.

Date de intrare:

- Obiect `Persoana` (nume, venit lunar, vechime în muncă, scor credit etc.)
- Suma dorită (`double suma`)
- Perioada (`int luni`)

Responsabili în chain:

1. `VerificatorBirouCredit`
 - Respinge dacă persoana este rău-platnic (ex: scor sub un prag).
2. `VerificatorVechime`
 - Respinge dacă vechimea în muncă este sub X luni.
3. `VerificatorGradIndatorare`
 - Calculează rata lunară și verifică dacă gradul de îndatorare depășește un procent (ex: 40% din venit).

Comportament:

- Fiecare responsabil:
 - Dacă regula NU este respectată → returnează `false` (lanțul se oprește)
 - Dacă regula este respectată → pasează cererea mai departe
- Dacă toate verificările sunt trecute → returnează `true`

Output:

`true` (credit aprobat) sau `false` (respins)

Problema 2 – Calcul credit maxim sau excepție

Context:

În loc să spună doar „da” sau „nu”, sistemul determină suma maximă pe care o poate primi clientul.

Cerință:

Să se implementeze un chain care:

- fie calculează suma maximă eligibilă
- fie aruncă o excepție dacă persoana nu este eligibilă deloc

Date de intrare:

- Obiect `Persoana`
- Perioada dorită

Responsabili posibili:

1. `VerificatorEligibilitateMinima`
 - Dacă nu îndeplinește condițiile de bază → aruncă excepție (`CreditRefuzatException`)
2. `LimitatorVechime`
 - Limitează suma maximă în funcție de vechime
3. `LimitatorVenit`
 - Calculează suma maximă în funcție de venit (ex: rata $\leq 40\%$ din venit)
4. `LimitatorScorCredit`
 - Ajustează suma în funcție de scorul de credit

Comportament:

- Fiecare handler:
 - Poate modifica suma maximă curentă
 - Sau poate opri chain-ul (excepție)
- Chain-ul pornește cu o valoare mare (ex: suma cerută sau un plafon general)
- Fiecare pas poate reduce această valoare

Output:

- Suma maximă eligibilă (`double`)
- Sau excepție dacă nu este eligibil

Problema 3 – Preprocesare text (filtrare în chain)

Context:

Se dorește procesarea unui text prin mai multe etape simple, fiecare aplicată de un handler.

Cerință:

Să se implementeze un chain care primește un `String` și aplică succesiv operații de preprocesare.

Date de intrare:

- `String text`

Idei de filtre (ușor de implementat în Java):

1. `LowerCaseFilter`
 - Transformă textul în litere mici
2. `RemoveExtraSpacesFilter`
 - Elimină spațiile multiple (ex: "ana are" → "ana are")
3. `TrimFilter`
 - Elimină spațiile de la început și final
4. `RemovePunctuationFilter`
 - Elimină semnele de punctuație (.,!? etc.)
5. `ReplaceDiacriticsFilter`
 - Înlocuiește diacriticele (ex: ă → a, ș → s)
6. `StopWordsFilter` (opțional, simplu)
 - Elimină cuvinte comune dintr-o listă mică (ex: „și”, „în”, „la”)
7. `ShortWordFilter`
 - Elimină cuvintele mai scurte de X caractere
8. `DuplicateWordRemover`
 - Elimină cuvintele duplicate

Comportament:

- Fiecare handler:
 - Primește textul
 - Îl modifică
 - Îl trimite mai departe
- Chain-ul returnează textul final procesat

Output:

- Textul procesat final

Problema 4 – Structură organizațională (angajați și departamente)

Context:

O companie este organizată în departamente, iar fiecare departament poate conține:

- angajați individuali
- sub-departamente

Cerință:

Să se implementeze un sistem bazat pe **Composite** pentru a reprezenta ierarhia organizațională.

Structură:

- `Angajat` (leaf)
- `Departament` (composite)

Date:

- `Angajat`: nume, salariu
- `Departament`: nume, listă de subordonati (angajați sau departamente)

Operații:

1. Calcul salariu total pentru un departament (include tot arborele)
2. Afișare structură ierarhică (indentare)
3. Număr total de angajați dintr-un departament

Comportament:

- `Angajat` → returnează propriul salariu / 1 angajat
- `Departament` → agregă valorile din copii

Problema 2 – Sistem de fișiere (foldere și fișiere)

Context:

Se dorește simularea unui sistem de fișiere similar cu cel dintr-un OS.

Cerință:

Să se implementeze un model Composite pentru:

- foldere (care pot conține fișiere și alte foldere)
- fișiere (elemente simple)

Structură:

- `Fisier` (leaf)
- `Folder` (composite)

Date:

- `Fisier`: nume, dimensiune (KB)
- `Folder`: nume, listă de elemente

Operații:

1. Calcul dimensiune totală folder
2. Afișare structură (tree view)
3. Căutare fișier după nume
4. Ștergere element dintr-un folder

Comportament:

- `Fisier` → dimensiune proprie
- `Folder` → sumă dimensiuni copii